

Phase 2: Advanced Research, Data Acquisition, and Architectural Expansion Report

Project: E-Learning System Ontology (v2)

Authors: Jalil Guliyev, Mahammadali Aliyev, Heydar Mehdizade

Date: May 14, 2026

Classification: Academic Technical Report - Phase 2 Final

1. Mandatory Paper Review: Behavioral Prediction and Deep Learning

1.1 Paper Overview

The mandatory research paper, "**Personalized Ontology and Deep Training Tree-Based Optimal GRU-RNN for Prediction of Students' Behavior**," addresses a critical gap in modern E-Learning systems: the inability to adapt to dynamic learner behaviors using static knowledge structures. The study proposes a hybrid neurosymbolic architecture that leverages the formal structure of ontologies alongside the predictive power of Gated Recurrent Unit - Recurrent Neural Networks (GRU-RNN).

1.2 Technical Methodology & Algorithmic Contributions

The paper introduces a highly optimized neural network architecture designed for time-series behavioral data. The core technical contributions include:

- Gated Recurrent Units (GRU):** Unlike standard RNNs, GRUs utilize update and reset gates to effectively manage long-term dependencies without the computational overhead of LSTMs. This is vital for real-time educational platforms tracking thousands of concurrent user interactions.
- Deep Training Tree (DTT):** The researchers implemented a DTT structure to resolve the "Vanishing Gradient" problem. This allows the network to maintain a deeper memory of a student's past performance (e.g., initial module scores) when predicting future outcomes (e.g., final exam success).
- Black Widow Optimization (BWO):** A bio-inspired meta-heuristic algorithm used to fine-tune the weights of the GRU-RNN. This prevented the common issue of delayed convergence and ensured a high **F-score of 97.4%** in classifying learner behaviors such as "Sedentary," "Aggressive," or "Cognitive."

1.3 Adaptation to Our E-Learning Project

We plan to integrate these findings by expanding our ABox from static records to temporal behavioral sequences. By extracting the time delta between :submissionDate and :dueDate across multiple assignments, our system can identify "Procrastination Patterns." These patterns, when fed into a GRU model, will allow our ontology to infer a new class: :AtRiskStudent, enabling instructors to intervene proactively before a student fails a course.

2. Research Integration: Automated Ontology Population

2.1 Study Selection: Ontology Population using LLMs

To resolve the "Knowledge Acquisition Bottleneck," we integrated methodologies from "**Ontology Population using LLMs**" (**Saki Norouzi et al.**). The study demonstrates that manual population of individuals (ABox) is not scalable for modern university environments where course catalogs and syllabi change every semester.

2.2 Implementation Strategy: Few-Shot Prompting

Our architectural strategy shifts from manual Turtle entry to an **LLM-Driven Extraction Pipeline**. By using Few-Shot Prompting, we provide a Large Language Model with a snippet of our elearning-tbox.ttl schema and a few examples of unstructured text mapped to Turtle triples.

The model then processes unstructured course descriptions and automatically generates ABox instances. This ensures that our ontology stays updated with the latest faculty changes and prerequisite shifts without requiring a Knowledge Engineer for every update.

3. Data Acquisition Pipeline: From Unstructured Text to Semantic Triples

The data acquisition process for Phase 2 follows a rigorous multi-stage pipeline designed to ensure data integrity and semantic alignment with our formal TBox.

Phase	Description	Technical Tools/Methods
Collection	Scraping university course catalogs and instructor syllabi.	Web Scraping / PDF Parsing (PyPDF2)
Preprocessing	Text normalization, removing noise, and tokenization.	NLP Cleaning Pipelines (NLTK/SpaCy)
Extraction	Identifying entities (Professors, Credits) and relationships.	LLM NER (Named Entity Recognition)
Serialization	Converting extracted data into valid Turtle (.ttl) syntax.	Python RDFLib / Prompt Engineering

3.1 Preprocessing and Mapping

Raw academic data is often messy. Our pipeline converts PDF-based syllabi into structured JSON formats before mapping them to the ontology. For instance, a sentence like *"Dr. Smith teaches CS501 and requires Math 101 as a prerequisite"* is decomposed into:

- `:Instructor_Smith rdf:type :Instructor .`
- `:Course_CS501 :isTaughtBy :Instructor_Smith .`
- `:Course_CS501 :hasPrerequisite :Course_Math101 .`

4. Ontology Usage and Architectural Expansion

4.1 SPARQL-Based Reasoning in Backend Services

Our ontology is designed to serve as a semantic backend. By utilizing a SPARQL endpoint, the application layer can perform complex logical checks that standard SQL databases cannot handle easily.

```
# SPARQL Query: Identify students who enrolled in courses without meeting prerequisites

SELECT ?student ?course ?missingPre
WHERE {
    ?student :enrolledIn ?course .
    ?course :hasPrerequisite ?missingPre .

    FILTER NOT EXISTS { ?student :submits ?sub . ?sub :submissionFor
        ?missingPre }

}
```

4.2 Expansion: SWRL and Linked Open Data (LOD)

To future-proof the system, we have identified two primary expansion paths:

- 1. **Semantic Web Rule Language (SWRL):** We will implement rules such as `Student(?s) ^ hasGrade(?s, ?g) ^ swrlb:greaterThan(?g, 90) -> HonorStudent(?s)`. This allows the reasoner to automatically manage student rankings without hard-coded application logic.
- 2. **LOD Integration:** By linking our Instructor individuals to **DBpedia** and **ORCID** URIs, we can enrich our local data with global academic publication records and citation counts, providing a richer experience for students.

5. Conclusion

Phase 2 has successfully transitioned the E-Learning System Ontology from a static conceptual model to a dynamic, predictive, and scalable architecture. By integrating GRU-RNN behavioral prediction and LLM-based data population, the system is now capable of handling the complexities of modern educational data while maintaining strict semantic integrity through a decoupled TBox/ABox design.